Name- Bhakare Purva Shivaji

Roll no-08

Practical no-8

```
# Decision Tree Classification
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import matplotlib.pyplot as plt
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Create Decision Tree model
model = DecisionTreeClassifier(random_state=42)
# Train model
model.fit(X_train, y_train)
# Predict
y_pred = model.predict(X_test)
# Calculate metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')
# Display results
print("Decision Tree Classifier")
```

```
print("------------------------")
print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)
# Create graph
metrics = ['Accuracy', 'Precision', 'Recall', 'F1 Score']
values = [accuracy, precision, recall, f1]
plt.bar(metrics, values)
plt.xlabel("Evaluation Metrics")
plt.ylabel("Score")
plt.title("Decision Tree Performance")
plt.ylim(0, 1.1)
plt.show()
```

Output-

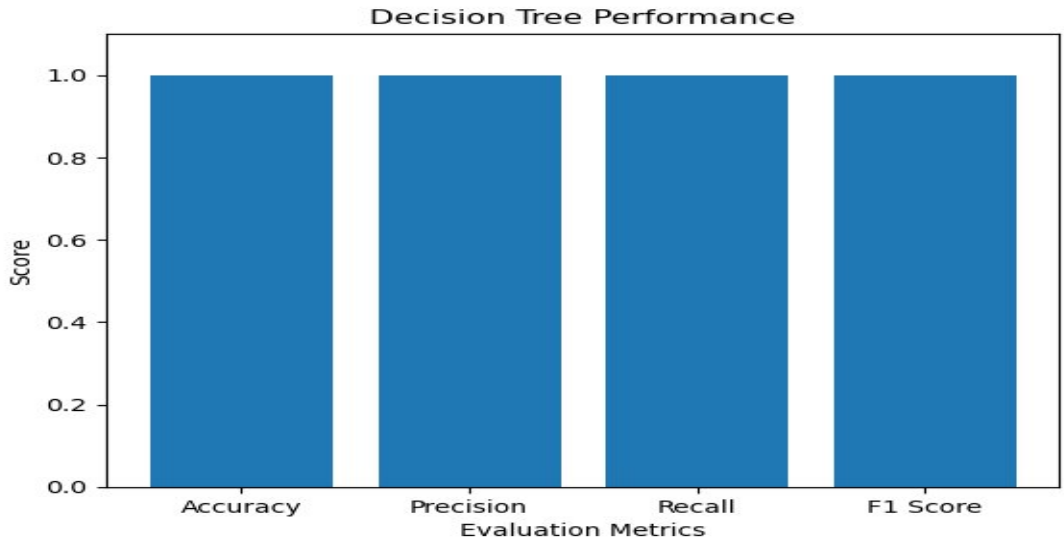
Decision Tree Classifier

Accuracy : 1.0

Precision: 1.0

Recall   : 1.0

F1 Score : 1.0



```
# Naive Bayes Classification


from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import matplotlib.pyplot as plt
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Create Naive Bayes model
model = GaussianNB()
# Train model
model.fit(X_train, y_train)
# Predict
y_pred = model.predict(X_test)
# Calculate metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')
# Display results
print("Naive Bayes Classifier")
print("----------------------")
print("Accuracy :", accuracy)
```

```
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)
# Create graph
metrics = ['Accuracy', 'Precision', 'Recall', 'F1 Score']
values = [accuracy, precision, recall, f1]
plt.bar(metrics, values)
plt.xlabel("Evaluation Metrics")
plt.ylabel("Score")
plt.title("Naive Bayes Performance")
plt.ylim(0, 1.1)
plt.show()
```

Output-

```
Naive Bayes Classifier
----------------------
Accuracy : 1.0
Precision: 1.0
Recall   : 1.0
F1 Score : 1.0
```

